Projeto 2
2025/2026

Engenharia Informática

Projeto 2

Biblioteca de artigos ciêntificos

Aluno:
Gilberto Parente 15330

Docentes:
Professor Miguel Cruz
Professor Pedro Faria
Professor Marcelo Antunes Fernandes
Professor David Verde

Sumário

1. Introdução *3*
1.1 Contextualização 3
1.2 Objetivos 3
2. Requisitos do Sistema a desenvolver *4*
2.1 Requisitos Funcionais – Autor 4
2.2 Requisitos Funcionais – Aluno / Leitor 4
2.3 Requisitos Funcionais – Administrador 5
2.4 Requisitos Não Funcionais 5
3. Modelo ER *6*
3.1 Modelo ER – descrição 6
3.2 Modelo ER - diagrama 10
4. Modelo de classes *11*
4.1 Modelo de classes - diagrama 11
5. Modelo de casos de uso *12*
5.1 Modelo de casos de uso – diagrama 12
6. Estado da aplicação *13*
7. Conclusão *15*
7.1 Conclusão 15

1. Introdução

1.1 Contextualização

No âmbito da Unidade Curricular de Projeto 2 do 2º Ano da licenciatura em Engenharia Informática no Instituto Politécnico de Viana do Castelo, foi proposta a implementação de uma aplicação que contemplasse a compra ou a pesquisa de artigos científicos, através de uma plataforma desktop e Web.
Com a elaboração deste projeto, os alunos têm a oportunidade de aprender e sobre tecnologias novas, desenvolvendo neste projeto conhecimentos sobre a temática de Engenharia de software, mas também sobre o processo de Engenharia reversa.

Como este Projeto não transita da Unidade Curricular do 1º ano, está a ser desenvolvido um projeto novo, o que neste caso, levou ao desenvolvimento da análise e construção da base de dados e do modelo de classe do início.

Como referido, este projeto é uma Biblioteca de artigos científicos, gerida por um Administrador e que como utilizadores terá dois tipos: Autores de artigos e leitores.
Este projeto trata-se do desenvolvimento de uma aplicação Desktop para utilizadores de BackOffice e uma aplicação Web para utilizadores.

A base de dados está alocada num servidor PostgreSQL. O acesso tabelas realiza-se pelas entidades de classes mapeadas através da tecnologia ORM - Object-Relational Mapping, usando Java Hibernate para persistir as classes na base de dados.

1.2 Objetivos

Objetivos:

- Identificar o ambito do projeto;
- Realizar a análise e tipificar os requisitos;
- Realizar o desenho dos casos de uso;
- Desenhar o medelo ER da base de dados;
- Desenhar o medelo de classes;

2. Requisitos do Sistema a desenvolver

2.1 Requisitos Funcionais – Autor

- Como Autor, quero registar-me na plataforma, para poder publicar artigos científicos.
- Como Autor, quero autenticar-me no sistema, para poder gerir os meus artigos.
- Como Autor, quero adicionar artigos na plataforma, para disponibilizar conteúdo científico aos utilizadores.
- Como Autor, quero definir se o artigo é pago ou gratuito, para controlar o acesso ao conteúdo.
- Como Autor, quero definir o preço do artigo, para poder comercializar o meu conteúdo.
- Como Autor, quero editar ou eliminar os meus artigos, para manter a informação atualizada.
- Como Autor, quero visualizar quantos utilizadores adquiriram os meus artigos, para avaliar a procura do conteúdo.
- Como Autor, quero visualizar os utilizadores que compraram os meus artigos, para analisar o interesse no meu trabalho.

2.2 Requisitos Funcionais – Aluno / Leitor

- Como Aluno, quero registar-me na plataforma, para poder aceder aos artigos científicos.
- Como Aluno, quero pesquisar artigos científicos, para encontrar conteúdos relevantes.
- Como Aluno, quero filtrar artigos por temática, para facilitar a procura de conteúdos de interesse.
- Como Aluno, quero comprar artigos pagos, para ter acesso ao conteúdo completo.
- Como Aluno, quero visualizar os artigos que adquiri, para poder consultá-los posteriormente.

2.3 Requisitos Funcionais – Administrador

- Como Administrador, quero registar utilizadores no sistema, para gerir quem pode aceder à plataforma.
- Como Administrador, quero editar dados de utilizadores, para manter a informação atualizada.
- Como Administrador, quero eliminar utilizadores, para remover contas indevidas ou inativas.
- Como Administrador, quero gerir autores do sistema, para controlar quem pode publicar artigos.
- Como Administrador, quero criar, editar e eliminar temáticas, para organizar os artigos científicos.
- Como Administrador, quero gerir os artigos do sistema, para garantir a qualidade dos conteúdos publicados.
- Como Administrador, quero gerir os pagamentos dos alunos e leitores, para fazer a gestão financeira da plataforma.

2.4 Requisitos Não Funcionais

- O sistema deve suportar múltiplos utilizadores simultaneamente sem degradação significativa de desempenho.
- O sistema deve garantir autenticação de utilizadores através de login e palavra-passe.
- O sistema deve proteger os dados dos utilizadores contra acessos não autorizados.
- Apenas administradores podem gerir utilizadores e temáticas.
- A aplicação deve possuir uma interface intuitiva e fácil de utilizar.
- Os utilizadores devem conseguir pesquisar artigos de forma simples e rápida.

3.Modelo ER

3.1 Modelo ER – descrição

Objetivo da Base de Dados

A base de dados foi concebida para suportar uma **biblioteca online de artigos científicos**, onde os utilizadores podem:

- consultar artigos científicos
- aceder a artigos gratuitos
- comprar artigos pagos
- visualizar autores dos artigos

O sistema também permite a gestão de utilizadores e compras.

Tabela - User

A tabela **User** representa os utilizadores registados na plataforma.

Campo	Tipo	Descrição
id_user	INTEGER (PK)	Identificador único do utilizador
name	VARCHAR(100)	Nome do utilizador
email	VARCHAR(100)	Endereço de email do utilizador (único)
password	VARCHAR(255)	Palavra-passe do utilizador
is_admin	BOOLEAN	Indica se o utilizador possui permissões de administrador

Observações

- **id_user** é a chave primária da entidade.
- O atributo **is_admin** permite distinguir administradores de utilizadores normais.

Tabela - Author

A tabela **Author** representa os autores responsáveis pela produção dos artigos científicos.

Campo	Tipo	Descrição
id_author	**INTEGER (PK)**	**Identificador único do autor**
id_user	**INTEGER (FK)**	**Utilizador associado ao autor**
affiliation	**VARCHAR(150)**	**Instituição ou organização à qual o autor pertence**

Observações

- Cada autor pode estar associado a vários artigos científicos.

Tabela Thematics

A tabela **Thematics** representa as áreas temáticas associadas aos artigos científicos.

Campo	Tipo	Descrição
id_thematic	INTEGER (PK)	Identificador único da temática
description	VARCHAR(100)	Descrição da área temática

Observações

- Existe um índice (idx_thematic_description) criado sobre o campo description para melhorar o desempenho das pesquisas por temática.

Tabela - Article

A tabela **Article** representa os artigos científicos disponíveis na biblioteca digital.

Campo	Tipo	Descrição
id_article	INTEGER (PK)	Identificador do artigo
title	VARCHAR	Título do artigo
abstract	TEXT	Resumo do artigo
publication_date	DATE	Data de publicação
price	DECIMAL	Preço do artigo
file_path	VARCHAR	Caminho do ficheiro do artigo

Observações

- Se o **preço for igual a zero**, o artigo é considerado **gratuito**.
- Caso contrário, o utilizador necessita de efetuar a compra para aceder ao artigo.

Tabela - Purchase

A entidade **Purchase** representa as compras realizadas pelos utilizadores.

Campo	Tipo	Descrição
id_purchase	INTEGER (PK)	Identificador da compra
id_user	INTEGER (FK)	Utilizador que realizou a compra
id_article	INTEGER (FK)	Artigo adquirido
purchase_date	DATE	Data da compra
amount	DECIMAL	Valor pago
paid	BOOLEAN	Indica se o pagamento foi efetuado

Observações

- **id_user** é uma chave estrangeira que referencia a entidade **User.**
- **id_article** é uma chave estrangeira que referencia a entidade **Article.**

Tabela - ArticleAuthor

A entidade **ArticleAuthor** é uma tabela associativa utilizada para representar a relação entre artigos e autores.

Campo	Tipo	Descrição
id_article	INTEGER (FK)	Identificador do artigo
id_author	INTEGER (FK)	Identificador do autor

Observações

- Esta entidade resolve uma relação **muitos-para-muitos (N:M)** entre artigos e autores.
- A chave primária pode ser composta por **(id_article, id_author).**
-

Relações entre Entidades

Relação User – Purchase

Esta relação representa as compras efetuadas pelos utilizadores.

Tipo de relação

1 : N

Descrição

Um utilizador pode realizar várias compras, mas cada compra pertence apenas a um utilizador.
User (1)-(N) Purchase

Relação Article – Purchase

Esta relação representa os artigos adquiridos pelos utilizadores.

Tipo de relação

1 : N

Descrição

Um artigo pode ser comprado por vários utilizadores, mas cada registo de compra refere-se apenas a um artigo.
Article (1)-(N) Purchase

Relação Article – Author

Esta relação representa a autoria dos artigos científicos.

Tipo de relação

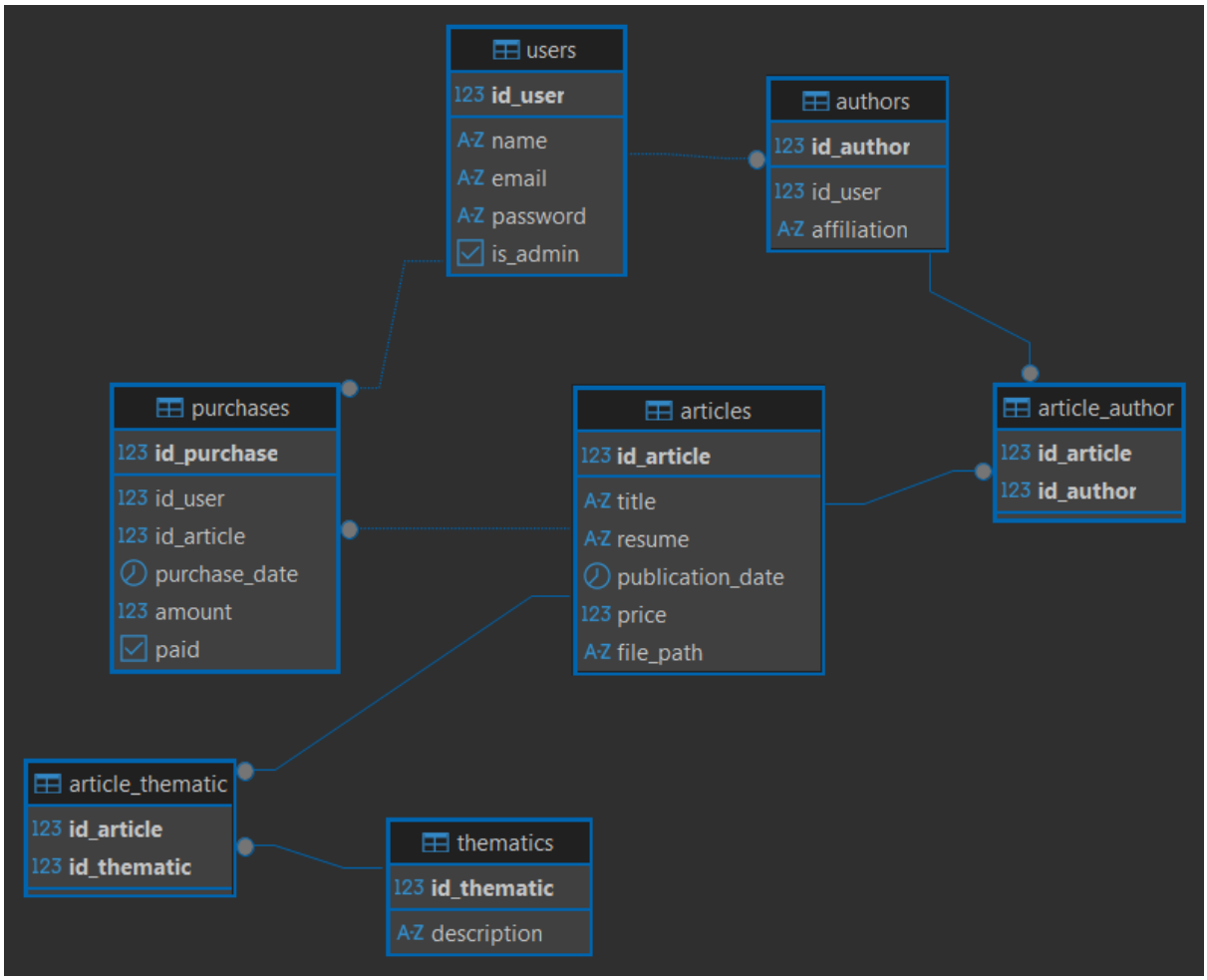
N : M

Descrição

- Um artigo pode ter vários autores.
- Um autor pode escrever vários artigos.

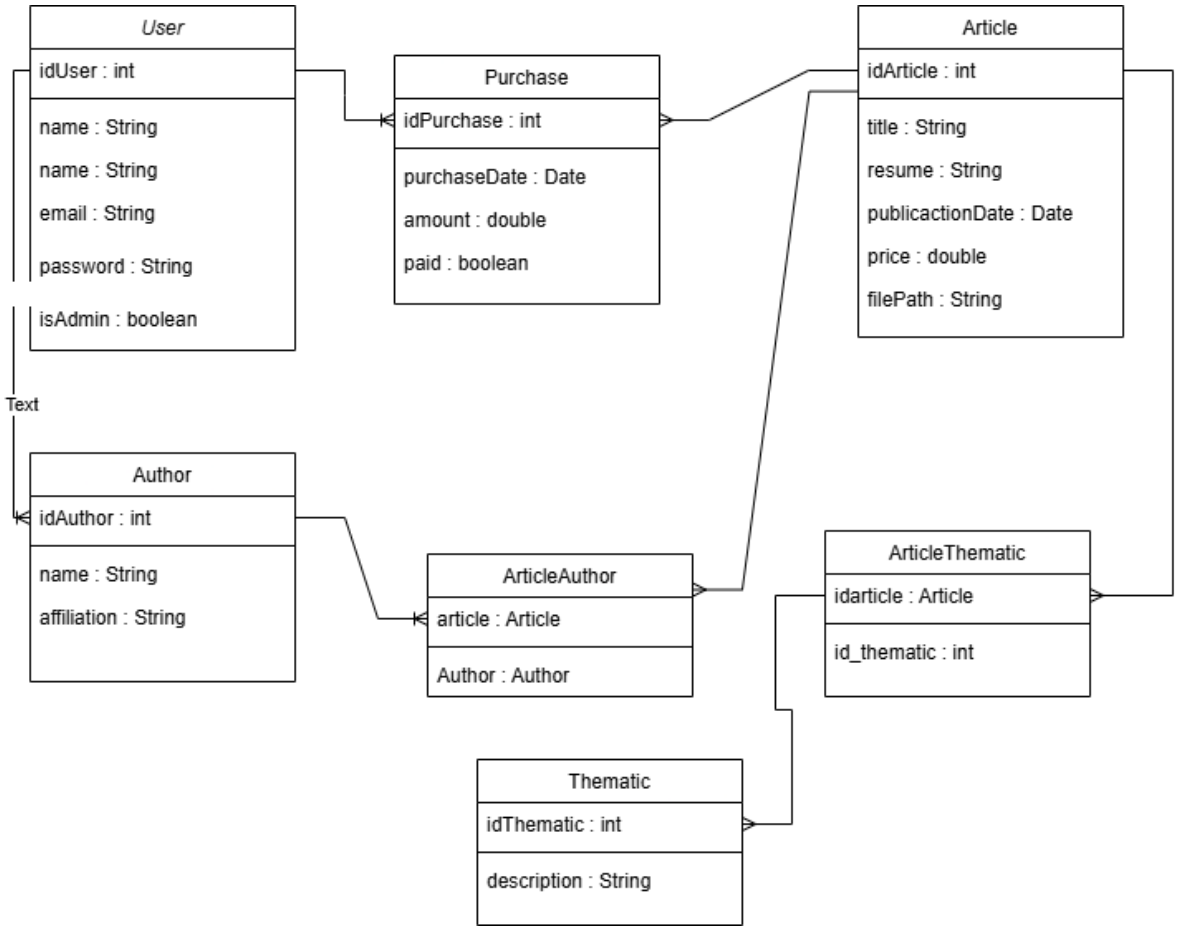
Esta relação é implementada através da entidade associativa **ArticleAuthor**.
Article (N)-(N) Author

3.2 Modelo ER - diagrama



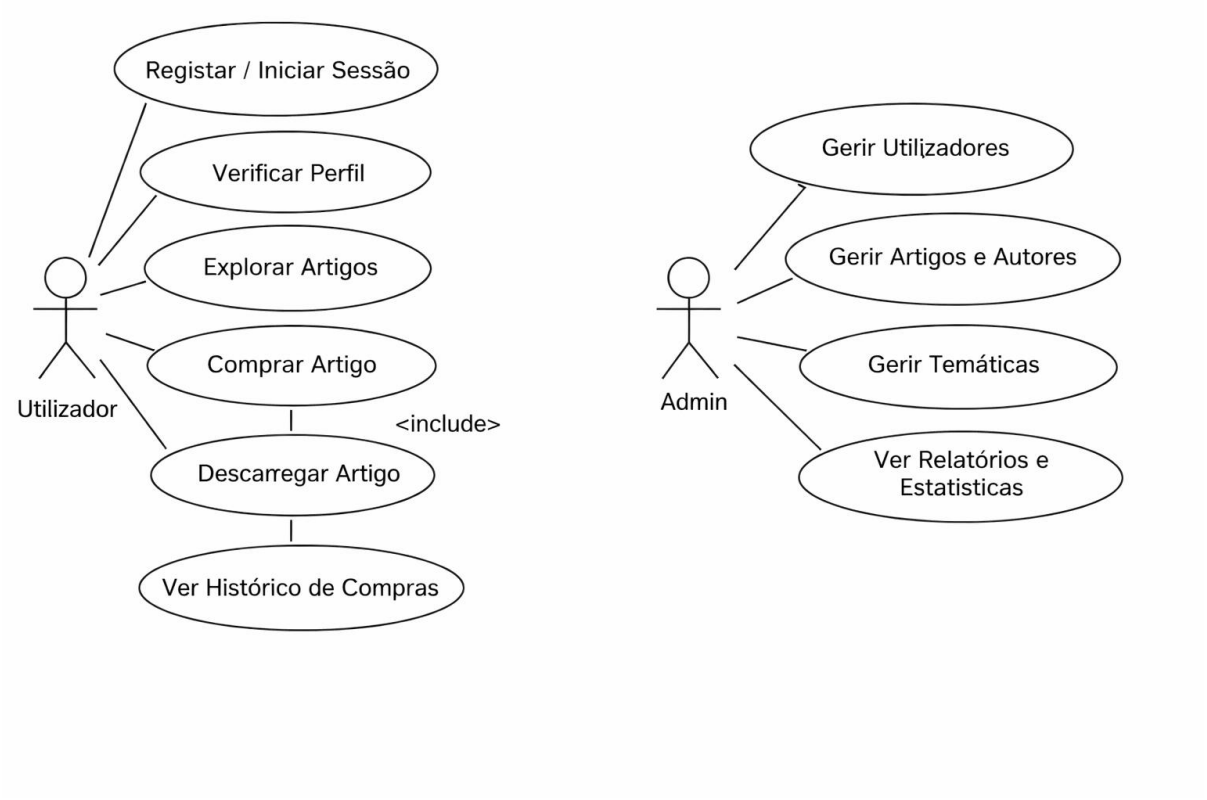
4.Modelo de classes

4.1 Modelo de classes - diagrama



5. Modelo de casos de uso

5.1 Modelo de casos de uso – diagrama



6.Estado da aplicação

O projeto **Scientific Library** foi estruturado seguindo uma arquitetura, composta por um **projeto principal (projeto-mãe)** e três subprojetos responsáveis por diferentes camadas da aplicação. Esta organização permite separara manutenção do código e permitir que diferentes interfaces utilizem a mesma lógica de negócio.
A estrutura atual do projeto é composta pelos seguintes módulos:

- Library - Projeto principal
- BLL – Business Logic Layer
- Web Application
- Desktop Application

Projeto Library
O **projeto** funciona como o ponto central da solução, sendo responsável pela organização dos diferentes módulos da aplicação.
Este projeto não contém lógica funcional da aplicação, mas define:

- a estrutura geral do sistema
- a gestão das dependências entre os módulos
- a organização do código em camadas

Projeto BLL – Business Logic Layer
O projeto **BLL** é responsável por toda a **lógica de negócio da aplicação** e pela **interação com a base de dados.**
Neste módulo foram implementados:

- **Entidades (Entities)**

As entidades representam as tabelas da base de dados e utilizam **JPA/Hibernate** para o mapeamento objeto-relacional.

- **Repositórios (Repositories)**

Foi implementado um **GenericRepository**, responsável por fornecer operações CRUD reutilizáveis para todas as entidades.
Operações disponíveis:
Com base neste repositório genérico foram criados repositórios específicos para determinadas entidades, permitindo adicionar funcionalidades mais especializadas quando necessário.

Configuração do Hibernate
A ligação à base de dados é gerida através da classe **HibernateUtil**, que cria e gere a SessionFactory.
Esta classe permite:

- abrir sessões de ligação à base de dados
- controlar transações
- executar operações de persistência através do Hibernate.

Validação da Camada BLL
Para validar o funcionamento da camada de lógica de negócio foi criada uma **classe Main de teste**, que permite interagir diretamente com a base de dados através de um menu em consola.
As operações atualmente implementadas permitem:

- inserir utilizadores
- listar utilizadores
- atualizar utilizadores
- eliminar utilizadores

Repositório Git:
https://github.com/gilbertoparente/Library.git

7.Conclusão

7.1 Conclusão

Nesta primeira etapa do projeto foi possível estabelecer as bases para o desenvolvimento da aplicação Scientific Library, através da definição da estrutura da base de dados, da implementação do modelo de entidades e da criação da camada de acesso a dados.

Inicialmente foi realizada a modelação da base de dados, definindo as tabelas necessárias para suportar o funcionamento da plataforma, incluindo utilizadores, autores, artigos, temáticas e compras. Foram também definidas as relações entre estas entidades, garantindo a integridade referencial através da utilização de chaves estrangeiras.

Posteriormente, foi implementado o mapeamento objeto-relacional utilizando Hibernate, permitindo representar as tabelas da base de dados através de classes Java. Este processo incluiu a criação das entidades correspondentes às tabelas, bem como a definição das relações entre elas através das anotações da Jakarta Persistence API (JPA).

Foi também desenvolvida a camada de acesso a dados, baseada num repositório genérico que permite realizar operações CRUD (Create, Read, Update e Delete) sobre as entidades do sistema. Esta abordagem facilita a reutilização de código e simplifica a implementação de funcionalidades futuras.

Para validar o funcionamento da aplicação e da ligação à base de dados, foi implementada uma classe de teste com interface em consola, permitindo realizar operações básicas sobre a entidade de utilizadores. Esta validação permitiu confirmar que a comunicação entre a aplicação e a base de dados está a funcionar corretamente. Adicionalmente, foram elaborados diagramas UML, nomeadamente o diagrama de classes e o diagrama de casos

de uso, que permitem representar de forma clara a estrutura do sistema e as principais funcionalidades previstas.

Em conclusão, esta primeira fase permitiu estabelecer uma infraestrutura sólida para o desenvolvimento do sistema, garantindo que os principais componentes técnicos estão corretamente implementados. Com esta base concluída, o projeto poderá evoluir para as próximas etapas, que incluirão o desenvolvimento completo da lógica de negócio e das interfaces de utilização nas aplicações web e desktop.